

Morning Cheat Sheet — read this and nothing else

Java, Spring Boot & DSA · Infosys · Saturday 26 September · Learn nothing new today — revision only.

45 minutes. Nothing else. No new material.

The 10 answers that carry this interview

1. **HashMap**: `hashCode()`, then a spread of `h ^ (h >>> 16)`, then bucket index `(n-1) & hash`. Collisions chain in a list; since Java 8 a bucket over 8 entries becomes a red-black tree. Capacity 16, load factor 0.75, so it resizes and rehashes at 12."
2. **equals/hashCode**: "Equal objects must have equal hash codes. Override `equals` without `hashCode` and the map looks in the wrong bucket and never finds the entry."
3. **N+1**: "One query for the list, then one per row for the relation. Find it by counting queries in the SQL log. Fix with `JOIN FETCH`, `@EntityGraph`, batch fetching or a DTO projection."
4. **@Transactional internal call**: "It works through a proxy. An internal `this.method()` bypasses the proxy, so no transaction starts." Rolls back on **unchecked** exceptions only. **The same is true of your AOP aspect.**
5. **Constructor injection**: "Fields can be final, dependencies cannot be missing, and I can test with `new Service(mock)` without a Spring context."
6. **Spring AOP**: "A pointcut selects the methods, an advice is the code that runs. Spring wraps the bean in a proxy — JDK dynamic proxy with an interface, CGLIB otherwise."
7. **Brute force first**: "Two nested loops is $O(n^2)$ — a HashMap makes the lookup $O(1)$, so one pass, $O(n)$. Trading space for time."
8. **Binary search**: `lo + (hi - lo) / 2` to avoid overflow, `while (lo <= hi)`, only on sorted data, $O(\log n)$.
9. **Microservices trade**: "You buy independent deployment, scaling and fault isolation; you pay with network failure, no distributed transaction and harder debugging."
10. **Index cost**: "Faster reads, but every insert and update maintains every index, so I add them deliberately."

DSA — the six patterns

- **Hashing** → counting, lookup, duplicates, two sum, anagram. *If you are searching inside a loop, a HashMap probably makes it one pass.*
- **Two pointers** → palindrome, reverse in place, pair-with-sum in a **sorted** array. $O(1)$ space.
- **Sliding window** → max sum of size k, longest substring without repeats. *Slide: add the new, drop the old.*
- **Fast and slow pointers** → middle of a list, cycle detection (Floyd's).
- **Stack** → balanced brackets, next greater element. Use `ArrayDeque`, **not** the legacy `Stack` class.

- **Binary search** → sorted data only.

Always: restate → ask edge cases → say the brute force and its cost → improve → write, talking → trace one example → **state time and space.**

Edge cases to ask about every time: null, empty, one element, duplicates, all-equal.

Complexity: ArrayList get $O(1)$, search $O(n)$ · LinkedList get $O(n)$ · HashMap $O(1)$ avg · TreeMap $O(\log n)$ · sorting $O(n \log n)$ · binary search $O(\log n)$.

Reverse a linked list — three pointers, save `next` before you flip:

```
next = curr.next; curr.next = prev; prev = curr; curr = next;
```

Java traps: `Integer` cache is $-128..127$, so `128 == 128` is **false** boxed ·

```
new String("a") != "a" · 0.1 + 0.2 != 0.3, money is BigDecimal.
```

Core Java

- **Overloading** compile time, different params · **overriding** runtime, subclass. Static methods are **hidden**, not overridden.
- **Abstract class** = shared state and code, one only · **interface** = contract, many.
- **HashMap** not thread safe · **Hashtable** locks the whole map, legacy · **ConcurrentHashMap** locks per bucket.
- **String immutable** — pool, security, thread safety, cached hash.
- **Checked** = compiler forces it · **unchecked** extends `RuntimeException`.
- **finally** always runs except `System.exit()`. Never **return** from it.
- **Fail-fast** throws `ConcurrentModificationException` — remove via `iterator.remove()`.

Java 8

- **Intermediate** lazy (`filter`, `map`) · **terminal** triggers (`collect`, `reduce`).
- `map` one-to-one · `flatMap` one-to-many then flattened.
- `groupBy(X, counting())` · `groupBy(X, summingDouble(Y))`.
- **Optional**: `map` / `orElse` / `orElseThrow`, not `isPresent` + `get`.
- ⚠ Group in **SQL** when the data is in the database, not in a stream over a million rows.

Spring Boot

- **IoC** = framework controls creation · **DI** = how it supplies dependencies.

- Beans are **singleton by default** → keep them **stateless**. A mutable field is a concurrency bug and, on a banking platform, an audit failure.
- `@Repository` also **translates database exceptions**.
- `@RestControllerAdvice` = one error shape. Log the detail, return a generic message.
- Auto-configuration** = `@ConditionalOnClass` + `@ConditionalOnMissingBean`; your bean wins.
- `@WebMvcTest` slice · `@SpringBootTest` full · `@MockBean` replaces a bean.

JPA and Hibernate

- JPA** spec · **Hibernate** implementation · **Spring Data JPA** the layer on top.
- `@ManyToOne` / `@OneToOne` default **EAGER** — set them **LAZY**. `@OneToMany` / `@ManyToMany` **LAZY**.
- Dirty checking** — a managed entity needs no `save()`.
- `@Enumerated(EnumType.STRING)`, never **ORDINAL**. Money is **BigDecimal**.
- REQUIRES_NEW** = the audit row survives when the main transaction rolls back.
- Liquibase** = versioned schema migrations, so the schema is reproducible per environment.

SQL

- Left-most prefix**: an index on `(status, created_at)` does not serve `created_at` alone.
- Index killers: a **function on the column**, a **leading %** in **LIKE**.
- EXPLAIN**: `type: ALL` = full scan. Check `key`, `rows`, `Extra`.
- WHERE** before grouping · **HAVING** after. **UNION ALL** is faster than **UNION**.
- Customers with no orders = `LEFT JOIN ... WHERE t.id IS NULL`.

Your stories — the numbers

Standard Bank South Africa, Service Online · 10,000+ corporate clients.

AOP audit system — `@Aspect` pointcuts, **50,000+ transactions a day**, no change to business logic. *Your best story: it is an architecture decision, not a feature.*

Camunda BPMN approval workflow — card processing time down **60%**.

JasperReports — 1,000+ monthly reports. **FileNet** for

documents. **Liquibase**

migrations. **Kafka and RabbitMQ** messaging. **JUnit**, **SIT/UAT**, **Bamboo CI/CD**.

The AOP follow-up to have ready: pointcut selects, advice runs; Spring uses a **proxy**, so an internal `this.method()` call does not trigger the aspect — same as `@Transactional`.

The three questions you must not fumble

"Tell me about yourself" → Now → What you built → **Why this role**. Under 2 minutes.

"You left in February — what have you been doing?" → Two or three sentences, the **real** reason, then **specifically** what you have done since. Calm, no apology. A seven-month gap with a clear story is fine; a mumbled one is not. **Rehearse this more than any technical answer.**

"You look like a frontend developer. Why backend?" → *"I did both on the same product.*

The problems I enjoyed most were backend — the workflow modelling, the AOP design, the data model. The frontend work made me better at designing APIs, but backend is where I want the depth."

Questions to ask them "What would I be working on in the first few months?" · "What does the stack look like — Spring

Boot version, cloud, CI/CD?" · "How does code review work in the team?" ·

"What is the biggest technical problem the team is dealing with right now?"

The last five minutes

- You are immediately available.** Most candidates serve 60–90 days. Say it.
- Give the why, not just the what.** "HashMap stores key-value pairs" is textbook; "O(1) because it hashes to a bucket instead of scanning" is an engineer.
- If you do not know something: say so, then reason out loud.** That has never cost anyone an offer. Bluffing has.
- In the coding round, **keep talking**. Silence reads as stuck.
- Sit up. Breathe out slowly. Smile before you speak.

Go get it.